Reading 3: Higher-Order Functions and Closures

CS 61A · Structure and Interpretation of Computer Programs · Week 3

1. Why environments matter

Most confusion about functions in Python comes from a single wrong assumption: that a function looks up names where it was called. It does not. A function looks up names where it was defined. Once that distinction is firm, closures stop being mysterious and become an obvious consequence of how frames are linked.

An environment is a sequence of frames. Each frame holds bindings from names to values and a parent pointer to the frame that encloses it. Evaluating a name means searching the current frame, then its parent, and so on until the global frame. If the search reaches the global frame without finding the name, Python raises a NameError.

2. Frames are created by calls

Every call to a user-defined function creates exactly one new frame. The frame's parent is not the caller's frame — it is the frame in which the function was defined. Python stores that frame on the function object itself when the def statement is evaluated.

```
def square(x):
    return x * x

square(4)   # creates frame f1, parent = Global
```

Here the distinction is invisible, because square was defined in the global frame and called from the global frame. Nested definitions are where it becomes visible.

3. Closures

A closure is a function together with the environment it captured at definition time. The canonical example returns a function from a function:

```
def make_adder(n):
    def adder(k):
        return k + n
    return adder

add3 = make_adder(3)
add3(4)     # 7
```

Calling make_adder(3) creates frame f1 with n bound to 3. The def statement inside it creates a function object whose parent is f1. When make_adder returns, f1 is no longer on the call stack, but it is still reachable from the returned function, so it is not discarded. Calling add3(4) creates frame f2 whose parent is f1, and the lookup of n succeeds one hop up the chain.

This is the whole trick. Nothing is copied; nothing is special-cased. The variable survives because something still points at the frame that holds it.

4. Functions as arguments

A higher-order function is any function that takes a function as an argument or returns one. Taking one is the more common direction in everyday code:

```
def compose(f, g):
    return lambda x: f(g(x))

inc = lambda x: x + 1
dbl = lambda x: x * 2
compose(inc, dbl)(5)    # 11
```

5. Common mistakes

Mistake	What actually happens
Parent frame is the caller	The parent is the frame where def ran. The caller is irrelevant to lookup.
Returning a function copies its variables	Nothing is copied. The function holds a pointer to the defining frame.
A returned inner function sees the latest value	It sees whatever the binding holds when the lookup happens, not when it was defined.
Lambdas behave differently from def	They do not. A lambda creates the same kind of function object with the same parent rule.

6. Practice before the midterm

1. Draw the environment diagram for make_adder(3) followed by add3(4). Label every parent pointer.

2. Explain in one sentence why f1 is not discarded when make_adder returns.

3. Write compose(f, g) without using lambda.

4. Predict the output of a counter built with a closure, then run it and account for any difference.